

Module 3 [Inheritance]: (for Week 4 [29-3/11-12/2025])

Problem Statement: You have to create an inheritance among **Father-->Son -->GrandSon** class. The **father** class has the following data members

```
class Father{
    private:
        int money;
    protected:
        int gold;
    public:
        int land;
};
```

Now write the **Son** and **GrandSon** classes with **private/protected/public** access modifier and do the following:

- i) Try to access **money, gold** and **land** from Son class
- ii) Try to access **money, gold** and **land** from GrandSon class
- iii) Find the values of money, gold and land when different access modifier is used in the following table
- iv) Display the sum of money, gold and land in Grandson class.

Class		In Son class			In GrandSon class		
Son	GrandSon	money	gold	land	money	gold	land
public	public	?	?	?	?	?	?
protected	public	?	?	?	?	?	?
private	public	?	?	?	?	?	?
public	protected	?	?	?	?	?	?
protected	protected	?	?	?	?	?	?
private	protected	?	?	?	?	?	?
public	private	?	?	?	?	?	?
protected	private	?	?	?	?	?	?
private	private	?	?	?	?	?	?

Code:

```
#include<bits/stdc++.h>
using namespace std;
class Father
{
    private: int money=100;
    protected: int gold=10;
    public: int land=500;
    /*/ Father()
    {
        money=100;
        gold=10;
        land=500;
        cout << "created" << endl;
    }
    int getmoney()
    {
        return money;
    }*/
};
class Son : public Father
{
    public:
        Son()
        {

        }
        void display()
        {
            //cout << getmoney() << endl;
            cout << gold << endl;
            cout << land << endl;
        }
};
class Grandson : public Son
{
    public:
        Grandson()
        {}
        void display()
        {
            //cout << getmoney() << endl;
            cout << gold << endl;
            cout << land << endl;
        }
        //int return sum
};
```

```

int main()
{
    Son s;
    s.display();

    Grandson g;
    g.display();

    return 0;
}

```

Topic 2 [Types of Inheritance]: Learn and Test different types of inheritance in C++. In each inheritance draw the class diagram with class chain and try to access the data members of bases classes from child classes.

i) Single inheritance

<pre> class A{ private: int x; protected: int y; public: int z; } </pre>	<pre> class B:public A{ //write public method to //access x,y & z } </pre>	<pre> int main(){ B b; //call methods of class B return 0; } </pre>
--	--	---

Code:

```

#include <iostream>
using namespace std;

class A {
private:
    int x;
protected:
    int y;
public:
    int z;
    A()
    {

    }

    A(int a, int b, int c)
    {
        x=a;
        y=b;
        z=c;
    }
}

```

```

    }
};

class B : public A {
public:
    void displayFromA(class A &obj) {
        cout << "Accessing from class B:" << endl;
        cout << "y (protected): " << obj.y << endl;
        cout << "z (public): " << obj.z << endl;
    }
    B()
    {

    }
};

int main() {
    A obj1(10,20,30);
    B obj2;
    obj2.displayFromA(obj1);
    return 0;
}

```

Multi-level inheritance

<pre> class A{ private: int x; protected: int y; public: int z; } </pre>	<pre> class B:public A{ } </pre>	<pre> class C:public B{ //write public //method to //access x,y & z } </pre>	<pre> int main(){ C c; //call //methods of //class C return 0 } </pre>
--	--------------------------------------	--	--

Code: #include<bits/stdc++.h>

```

using namespace std;
class A{
private:
int x=10;
protected:
int y=11;
public:
int z=12;

```

```

int getx()
{
    return x;
}
};
class B:public
A{
};
class C:private
B{
public:
int gety()
{
    return y;
}
int getz()
{
    return z;
}
int getpvt()
{
    getx();
}
};
int main(){
C c;
cout << c.getpvt() << endl;
cout << c.gety() << endl;
cout << c.getz()<< endl;
return 0;
}

```

Multiple inheritance

<pre> class A{ private: int x; protected: int y; public: int z; } </pre>	<pre> class B{ private: int p; protected: int q; public: int r; } </pre>	<pre> class C:public A, Public B{ //write public method //to access //x,y,z,p,q & r } </pre>	<pre> int main(){ C c; //call //methods of //class C return 0 } </pre>
--	--	--	--

Code:

```
#include<bits/stdc++.h>
using namespace std;
class A{
private:
int x=1;
protected:
int y=2;
public:
int z=3;
int getx()
{
    return x;
}
};
class B{
private:
int p=11;
protected:
int q=22;
public:
int r=33;
int getp()
{
    return p;
}
};
class C:public A,public B
{
    public:
        int gety()
        {
            return y;
        }
        int getq()
        {
            return q;
        }
};

int main(){
    C c;
    cout << " x " <<c.getx() << endl;
    cout << " y " <<c.gety() << endl;
    cout << " z " << c.z<< endl;
```

```

cout << " p " << c.getp()<< endl;
cout << " q " << c.getq() << endl;
cout << " r " << c.r<< endl;
return 0;
}

```

Heirarchical inheritance

<pre> class A{ private: int x; protected: int y; public: int z; } </pre>	<pre> class B:public A { //write public method to access x,y & z } </pre>	<pre> class C:public A { //write method public to access x,y & z } </pre>	<pre> int main(){ B b; C c; //call //methods of //class B & C return 0 } </pre>
--	---	---	---

Code:

```

#include <bits/stdc++.h>
using namespace std;

```

```

class A {
private:
    int x;
protected:
    int y;
public:
    int z;

    A() {
        x = 7;
        y = 8;
        z = 9;
    }

    int getX() {
        return x;
    }
};

```

```

class B : public A {
public:

```

```

    void showB() {
        cout << "B: x = " << getX() << ", y = " << y << ", z = " << z << endl;
    }
};

class C : public A {
public:
    void showC() {
        cout << "C: x = " << getX() << ", y = " << y << ", z = " << z << endl;
    }
};

int main() {
    B b;
    C c;
    b.showB();
    c.showC();
    return 0;
}

```

Hybrid (Diamond) inheritance [virtual class]

<pre> class A{ private: int x; protected: int y; public: int z; } </pre>	<pre> class B:public A { } </pre>	<pre> class C:public A { } </pre>	<pre> class D:public B, public C { //write public method to access x,y & z } </pre>	<pre> int main(){ D d; //call //methods of //class D return 0 } </pre>
--	-----------------------------------	-----------------------------------	---	--

Code:

```

#include <bits/stdc++.h>
using namespace std;

```

```

class A {
private:
    int x;
protected:
    int y;
public:
    int z;

    A() {
        x = 11;
        y = 22;
    }
}

```

```

        z = 33;
    }

    int getX() {
        return x;
    }
};

class B : virtual public A {
};

class C : virtual public A {
};

class D : public B, public C {
public:
    void show() {
        cout << "x = " << getX() << endl;
        cout << "y = " << y << endl;
        cout << "z = " << z << endl;
    }
};

int main() {
    D d;
    d.show();
    return 0;
}

```

Topic 3 [Constructor & Destructor in inheritance]: Write the constructors & destructors for different types of inheritance are given as follows. Also follow and write the sequence of their execution.

i) Single inheritance

<pre> class A{ private: int ax; public: //write constructor to initialize ax //Write destructor } </pre>	<pre> class B:public A{ private: int bx; public: //write constructor to initialize bx //Write method to sum ax and bx //Write destructor } </pre>	<pre> int main(){ B b; //call methods of class B return 0; } </pre>
--	---	---

```

#include <bits/stdc++.h>
using namespace std;

```

```

class A {
protected:
    int ax;
public:
    A() {
        ax = 10;
        cout << "A constructor\n";
    }
    ~A() {
        cout << "A destructor\n";
    }
};

```

```

class B : public A {
private:
    int bx;
public:
    B() {
        bx = 20;
        cout << "B constructor\n";
    }
    void sum() {
        cout << "Sum = " << ax + bx << endl;
    }
    ~B() {
        cout << "B destructor\n";
    }
};

```

```

int main() {
    B b;
    b.sum();
    return 0;
}

```

ii) Multi-level inheritance

<pre>private: int ax; public: //write constructor to initialize ax //Write destructor }</pre>	<pre>class B:public A { private: int bx; public: //write constructor to initialize bx //Write destructor }</pre>	<pre>class C:public B { private: int cx; public: //write constructor to initialize cx //Write method to sum ax, bx and cx //Write destructor }</pre>	<pre>int main(){ C c; //call //methods of //class C return 0 }</pre>
---	--	--	--

```

#include <bits/stdc++.h>
using namespace std;

class A {
protected:
    int ax;
public:
    A() {
        ax = 10;
        cout << "A constructor\n";
    }
    ~A() {
        cout << "A destructor\n";
    }
};

class B : public A {
protected:
    int bx;
public:
    B() {
        bx = 20;
        cout << "B constructor\n";
    }
    ~B() {
        cout << "B destructor\n";
    }
};

class C : public B {
private:
    int cx;
public:
    C() {
        cx = 30;
        cout << "C constructor\n";
    }
    void sum() {
        cout << "Sum = " << ax + bx + cx << endl;
    }
    ~C() {
        cout << "C destructor\n";
    }
};

int main() {
    C c;
    c.sum();
}

```

```

    return 0;
}

```

Multiple inheritance

<pre> private: int ax; public: //write constructor to initialize ax //Write destructor } </pre>	<pre> class B{ private: int bx; public: //write constructor to initialize bx //Write destructor } </pre>	<pre> class C:public A, Public B{ private: int cx; public: //write constructor to initialize cx //Write method to sum ax, bx and cx //Write destructor } </pre>	<pre> int main(){ C c; //call //methods of //class C return 0 } </pre>
---	--	---	--

```

#include <bits/stdc++.h>
using namespace std;

class A {
protected:
    int ax;
public:
    A() {
        ax = 10;
        cout << "A constructor\n";
    }
    ~A() {
        cout << "A destructor\n";
    }
};

class B {
protected:
    int bx;
public:
    B() {
        bx = 20;
        cout << "B constructor\n";
    }
    ~B() {
        cout << "B destructor\n";
    }
};

class C : public A, public B {
private:

```

```

    int cx;
public:
    C() {
        cx = 30;
        cout << "C constructor\n";
    }
    void sum() {
        cout << "Sum = " << ax + bx + cx << endl;
    }
    ~C() {
        cout << "C destructor\n";
    }
};

int main() {
    C c;
    c.sum();
    return 0;
}

```

Heirarchical inheritance

<pre> class A{ private: int ax; public: //write constructor to initialize ax //Write destructor } </pre>	<pre> class B:public A { private: int bx; public: //write constructor to initialize bx //Write destructor } </pre>	<pre> class C:public A { private: int cx; public: //write constructor to initialize cx //Write method to sum ax, bx and cx //Write destructor } </pre>	<pre> int main(){ B b; C c; //call //methods of //class B & C return 0 } </pre>
--	--	--	---

```

#include <bits/stdc++.h>
using namespace std;

class A {
protected:
    int ax;
public:
    A() {
        ax = 10;
        cout << "A constructor\n";
    }
}

```

```

    }
    ~A() {
        cout << "A destructor\n";
    }
};

class B : public A {
private:
    int bx;
public:
    B() {
        bx = 20;
        cout << "B constructor\n";
    }
    ~B() {
        cout << "B destructor\n";
    }
};

class C : public A {
private:
    int cx;
public:
    C() {
        cx = 30;
        cout << "C constructor\n";
    }
    void sum() {
        cout << "Sum = " << ax + cx << endl;
    }
    ~C() {
        cout << "C destructor\n";
    }
};

int main() {
    B b;
    C c;
    c.sum();
    return 0;
}

```

Hybrid (Diamond) inheritance [virtual class]

<pre> class A{ private: int ax; public: //write constructo r to initialize ax //Write destructor } </pre>	<pre> class B:public A { private: int bx; public: //write constructo r to initialize bx //Write destructor } </pre>	<pre> class C:public A { private: int cx; public: //write constructor to initialize cx //Write destructor } </pre>	<pre> class D:public B, public C { private: int dx; public: //write constructor to initialize dx //Write method to sum ax, bx cx and dx //Write destructor } </pre>	<pre> int main(){ D d; //call //methods of //class D return 0 } </pre>
---	---	--	---	--

```

#include <bits/stdc++.h>
using namespace std;

class A {
protected:
    int ax;
public:
    A() {
        ax = 10;
        cout << "A constructor\n";
    }
    ~A() {
        cout << "A destructor\n";
    }
};

class B : virtual public A {
protected:
    int bx;
public:
    B() {
        bx = 20;
        cout << "B constructor\n";
    }
    ~B() {
        cout << "B destructor\n";
    }
};

class C : virtual public A {
protected:
    int cx;
public:

```

```

    C() {
        cx = 30;
        cout << "C constructor\n";
    }
    ~C() {
        cout << "C destructor\n";
    }
};

class D : public B, public C {
private:
    int dx;
public:
    D() {
        dx = 40;
        cout << "D constructor\n";
    }
    void sum() {
        cout << "Sum = " << ax + bx + cx + dx << endl;
    }
    ~D() {
        cout << "D destructor\n";
    }
};

int main() {
    D d;
    d.sum();
    return 0;
}

```